

VEDAIRO™

Enterprise 5.0.0

COMPLETE USER & OPERATIONS GUIDE

A practical guide to installation, configuration, administration, development, security, integrations, AI, commerce and production operations.

VEDAIRO™ Ecosystem License

Copyright (c) 2026 Cloud Soft Web LLP

Developed at: Cloud Soft Web Lab (WB, India)

Author & Principal Architect: Susanta Ghosh (CEO, Cloud Soft Web LLP)

Release documentation • v5.0.0

VEDAIRO™ Ecosystem License

Copyright (c) 2026 Cloud Soft Web LLP

Developed at: Cloud Soft Web Lab (WB, India)

Author & Principal Architect: Susanta Ghosh (CEO, Cloud Soft Web LLP)

1. Ownership

VEDAIRO™, its framework architecture, source-code organization, original documentation, branding and related ecosystem materials are proprietary intellectual property of Cloud Soft Web LLP unless a specific component or dependency is expressly identified as third-party/open-source software.

2. Permitted use

Subject to the terms supplied with a particular VEDAIRO distribution or commercial agreement, a licensee may install, configure, develop and deploy applications using VEDAIRO for its authorized business or project purposes.

3. Restrictions

No license is granted to misrepresent VEDAIRO as an independently authored framework, remove required copyright or attribution notices, redistribute proprietary VEDAIRO source as a competing framework, or sublicense rights beyond the applicable agreement.

4. Third-party services

Payment gateways, AI providers, SMTP providers, Redis, S3-compatible storage, databases and other external services remain subject to their own accounts, credentials, terms and licenses. VEDAIRO provides integration interfaces; it does not create provider accounts or fabricate secrets.

5. Security and deployment responsibility

The operator is responsible for production credentials, infrastructure, backups, access control, legal compliance, provider configuration and validation of live integrations. Never publish API keys or production secrets.

6. No warranty

To the maximum extent permitted by applicable law, the software and documentation are provided without an implied warranty of uninterrupted operation or fitness for a particular purpose. Production deployments should be tested in a staging environment.

7. License versioning

A future commercial agreement, enterprise contract or explicit LICENSE file distributed with a particular release may supersede this summary. This page is an ecosystem-license notice and not a substitute for a signed commercial agreement.

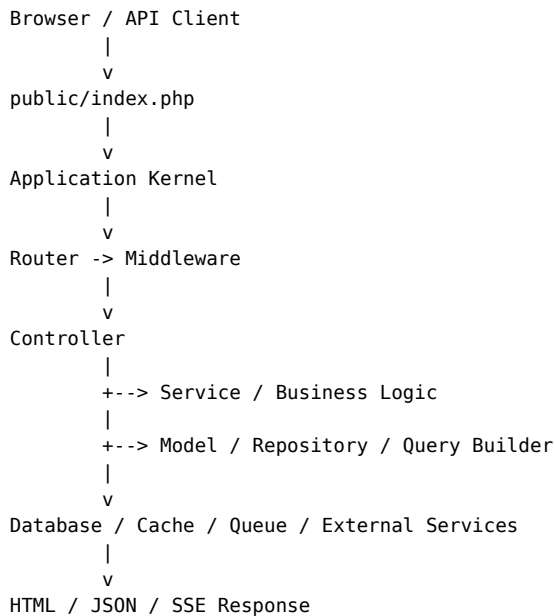
License note: This document is a practical license template/notice and should be reviewed by qualified legal counsel before commercial distribution.

Table of Contents

1. What VEDAIRO Enterprise is
2. Architecture and folder structure
3. Requirements and installation
4. Environment and database configuration
5. Apache / Nginx / .htaccess
6. CLI and first-run workflow
7. Routing, MVC and application flow
8. Database, ORM, migrations and pagination
9. CRUD, AJAX, search and uploads
10. Sessions, authentication, 2FA and RBAC
11. REST API, tokens, OpenAPI and rate limiting
12. Cart, checkout, orders, tax, coupons and invoices
13. Payments and webhooks
14. Mail and notifications
15. Cache, Redis, queue and scheduler
16. Storage: local and S3-compatible
17. Multi-tenancy and audit
18. AI engine, providers, RAG and agents
19. Real-time SSE and metrics
20. Internationalization
21. Backup, maintenance and production deployment
22. Security checklist
23. Testing and troubleshooting
24. Developer workflow and examples
25. Feature/status matrix and ecosystem boundaries

1. What VEDAIRO Enterprise is

VEDAIRO Enterprise 5.0.0 is a modular PHP 8.3+ framework/application foundation for MVC web applications, database systems, REST APIs, commerce workflows, infrastructure integrations and AI applications. The architecture separates HTTP handling, routing, middleware, controllers, services/models, persistence and response generation.



What this guide covers

- Installing and running VEDAIRO locally or on Apache/Nginx.
- Configuring MySQL/MariaDB, sessions, cache, mail, storage, payments and AI.
- Using CRUD, pagination, AJAX, cart and checkout.
- Operating authentication, TOTP 2FA, RBAC, API tokens and tenancy.
- Running queues, scheduler, backups and production health checks.
- Building applications on the framework without hard-coding infrastructure providers.

Important ecosystem boundary

External services are integrations. VEDAIRO can store configuration, validate connections, call APIs and verify webhooks, but provider accounts, servers and credentials must be supplied by the operator.

2. Architecture and folder structure

```
vedaïro/
├─ app/
├─ config/
├─ core/
│   ├─ Auth/
│   ├─ Authorization/
│   ├─ Cache/
│   ├─ Console/
│   ├─ Database/
│   ├─ Events/
│   ├─ Mail/
│   ├─ Notifications/
│   ├─ Queue/
│   ├─ Schedule/
│   ├─ Storage/
│   └─ Tenancy/
└─ ...
```

```

├─ database/
│  └─ migrations/
│  └─ seeders/
│  └─ factories/
├─ docs/
├─ public/
│  └─ index.php
│  └─ install.php
│  └─ .htaccess
├─ resources/
├─ routes/
│  └─ web.php
│  └─ api.php
├─ storage/
├─ .env
├─ .env.enterprise.example
├─ composer.json
└─ vedairo

```

Public document root

Only the `public/` directory should be exposed by the web server. Application source, `.env`, storage and configuration files should remain outside the public document root.

Recommended deployment separation

- `public/`: web-accessible entry point and static assets.
- `core/`: framework internals.
- `app/`: project-specific application code.
- `database/`: schema and seed data.
- `storage/`: logs, cache, uploads, backups and runtime data.
- `config/`: non-secret configuration defaults.
- `.env` / secret manager: credentials and environment-specific secrets.

3. Requirements and installation

Recommended requirements

Component	Recommended
PHP	8.3 or newer
Database	MySQL 8+ or MariaDB 10.6+
Web server	Apache 2.4+ or Nginx
CLI	PHP CLI enabled
Browser	Current Chrome, Firefox or Edge
Optional infrastructure	Redis, S3-compatible storage, SMTP

Local installation

```

# enter the project directory
cd VEDAIRO

# create environment file
cp .env.enterprise.example .env

# create database first, then configure .env

php vedairo migrate
php vedairo db:seed
php vedairo test

```

```
php vedairo serve 8000
```

On Windows, use an equivalent copy command or copy the example file manually. If Composer dependencies are required by a selected extension, run Composer in the project environment before using that extension.

Open the application

```
http://127.0.0.1:8000
```

Installer

The public/install.php entry point is provided for installation workflows. After a successful production installation, remove or disable public/install.php so that it cannot be reused by an attacker.

4. Environment and database configuration

Example environment

```
APP_ENV=local
APP_DEBUG=true
APP_URL=http://127.0.0.1:8000
APP_KEY=CHANGE_THIS_TO_A_LONG_RANDOM_SECRET

DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=vedairo
DB_USERNAME=root
DB_PASSWORD=

SESSION_DRIVER=file
CACHE_DRIVER=file

REDIS_HOST=127.0.0.1
REDIS_PORT=6379

MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USERNAME=
MAIL_PASSWORD=
MAIL_ENCRYPTION=tls

AI_PROVIDER=openai
OPENAI_API_KEY=
OPENAI_BASE_URL=https://api.openai.com/v1
OPENAI_MODEL=

S3_ENDPOINT=
S3_BUCKET=
S3_ACCESS_KEY=
S3_SECRET_KEY=
```

Database workflow

```
php vedairo migrate
php vedairo db:seed
```

Migration files are SQL files under database/migrations/. Executed migration filenames are tracked in the migration table so the same migration is not normally applied twice.

Production database principle

- Use a least-privilege database user.
- Do not use the MySQL root account for the application.
- Back up before destructive schema changes.
- Test restore procedures, not just backup creation.

5. Apache / Nginx / .htaccess

Apache

```
DocumentRoot /var/www/vedairo/public

<Directory /var/www/vedairo/public>
    AllowOverride All
    Require all granted
</Directory>
```

.htaccess concept

```
RewriteEngine On

RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [L]
```

Nginx

```
server {
    root /var/www/vedairo/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }
}
```

The exact PHP-FPM socket/path varies by operating system and PHP installation. Keep the application source outside the web root.

6. CLI and first-run workflow

Command	Purpose
php vedairo about	Show framework/runtime information
php vedairo route:list	List registered routes
php vedairo migrate	Run pending migrations
php vedairo db:seed	Load seed data
php vedairo queue:work	Start queue worker
php vedairo queue:work --once	Process one queue cycle
php vedairo backup [file]	Create a backup artifact
php vedairo test	Run framework checks
php vedairo serve 8000	Start local development server
php vedairo schedule:run	Execute registered scheduled tasks

Recommended first-run sequence

```
php vedairo about
php vedairo migrate
php vedairo db:seed
php vedairo test
php vedairo serve 8000
```

Do not run seeders blindly against a production database. Seeders may create sample records or default roles that are inappropriate for a live system.

7. Routing, MVC and application flow

Routes

```
Route::get('/products', 'ProductController@index');
Route::post('/products', 'ProductController@store');
Route::put('/products/{id}', 'ProductController@update');
Route::delete('/products/{id}', 'ProductController@destroy');
```

Request lifecycle

A request enters `public/index.php`, is bootstrapped by the kernel, matched by the router, processed through middleware, dispatched to a controller, passed to services/models as required, and returned as HTML, JSON or an SSE response.

Controller responsibility

- Accept request input.
- Call validation.
- Invoke application/service logic.
- Return a view or structured response.

Keep business rules out of controllers when they can be expressed as reusable services or domain operations.

8. Database, ORM, migrations and pagination

Query Builder

```
$products = Product::query()
    ->whereEq('status', 1)
    ->orderBy('id', 'DESC')
    ->paginate(
        20,
        (int)$request->input('page', 1),
        '/products'
    );
```

Pagination data

- `$p->items` — current records.
- `$p->total` — total matching records.
- `$p->page` — current page.
- `$p->perPage` — records per page.
- `$p->lastPage()` — final page number.
- `$p->links()` — rendered pagination links.

Database safety

Use parameterized queries or the Query Builder. Never concatenate untrusted request parameters into SQL. Validate sort columns against an allow-list rather than accepting arbitrary SQL fragments.

Transactions

```
DB::transaction(function () use ($order) {
    // create order
    // create order items
    // deduct stock
    // write audit/event records
});
```

Commerce operations that must succeed or fail together should be transactional.

9. CRUD, AJAX, search and uploads

Reference Product CRUD

Method	Endpoint	Purpose
GET	/products	List, search, filter and paginate
POST	/products	Create
PUT	/products/{id}	Update
DELETE	/products/{id}	Delete

AJAX

```
$.ajax({
  url: '/products',
  method: 'POST',
  headers: {
    'X-CSRF-TOKEN':
      $('meta[name="csrf-token"]').attr('content')
  },
  data: $('#productForm').serialize()
});
```

Uploads

- Validate MIME type, extension, size and dimensions.
- Generate a server-side filename.
- Store uploads outside executable source locations.
- Never trust the original filename.
- Use object storage for high-volume production deployments.

10. Sessions, authentication, 2FA and RBAC

Session authentication

```
Auth::login($user);
Auth::check();
Auth::user();
Auth::logout();
```

Session identifiers should be regenerated after authentication. Use secure, HttpOnly and SameSite cookies in production.

TOTP 2FA

VEDAIRO includes a TOTP implementation using 30-second SHA-1 codes and recovery codes. Enforce 2FA for privileged accounts and protect recovery codes like passwords.

RBAC

```
User
|
+--> Role
    |
    +--> Permission
```

Examples:
products.view
products.create
products.update
products.delete

reports.view

Authorization principle

Authentication answers “who are you?” Authorization answers “what are you allowed to do?”. Always perform authorization checks server-side; never rely on hiding a button in the UI.

11. REST API, tokens, OpenAPI and rate limiting

Health endpoint

GET /api/v1/health

Products

GET /api/v1/products?page=1&per_page=20&q=phone

Token

POST /api/v1/auth/token

Authorization: Bearer YOUR_TOKEN

Bearer tokens should be transmitted only over HTTPS. Store token material safely and revoke compromised credentials.

OpenAPI

The framework includes an OpenAPI endpoint/foundation for documenting API contracts. Keep the published schema synchronized with actual routes, authentication and response formats.

Rate limiting

Rate-limit login, token issuance, password reset, expensive AI requests and public API endpoints. Use Redis or another shared store for horizontally scaled deployments.

12. Cart, checkout, orders, tax, coupons and invoices

Cart

```
GET    /cart
POST   /cart/add
PUT     /cart/{id}
DELETE /cart/{id}
DELETE /cart
```

CartService stores product IDs and quantities in session state. At checkout, VEDAIRE must re-read prices and stock from the database and recalculate totals on the server.

Checkout sequence

```
Cart
-> Validate customer/address
-> Re-read prices
-> Validate stock
-> Calculate tax/discount/shipping
-> Create order
-> Create order items
-> Deduct stock
-> Create payment transaction
-> Commit
```

-> Send notification / queue follow-up

Tax, coupons and invoice

Tax and coupon services centralize business calculations. Invoice generation should use the final committed order values, not values supplied by the browser.

For financial applications, define rounding, tax jurisdiction, refund and cancellation policies explicitly and test them with representative cases.

13. Payments and webhooks

Payment architecture

Order

- > Payment Intent / Gateway Order
- > Customer pays at provider
- > Provider webhook
- > Signature verification
- > Idempotency check
- > Persist event
- > Reconcile provider status
- > Fulfil order / issue invoice

Supported integration direction

- Stripe adapter.
- Razorpay adapter.
- PaymentGateway abstraction.
- Refund interface.
- Webhook event persistence and signature-verification helpers.

Critical production rule

Never mark a payment as successful merely because the browser redirected to a success page. The server must verify the provider webhook/signature and reconcile the provider transaction state.

Provider credentials, merchant accounts and live settlement environments remain operator-owned.

14. Mail and notifications

SMTP

```
MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USERNAME=...
MAIL_PASSWORD=...
MAIL_ENCRYPTION=tls
```

Test SMTP delivery in staging. For production, use a dedicated sender domain, SPF/DKIM/DMARC where applicable, bounce monitoring and provider-specific limits.

Notifications

- Email notification abstraction.
- Database notification extension point.
- Webhook/external channel extension points.
- Queue long-running or bulk notifications.

15. Cache, Redis, queue and scheduler

Redis

```
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
```

Redis can be used for shared cache and queue/session strategies. In a multi-server deployment, use a centralized Redis service and secure network access.

Queue worker

```
php vedairo queue:work
```

Run queue workers under Supervisor, systemd or another process supervisor. Workers should be restarted safely during deployments and monitored for failures.

Scheduler

```
php vedairo schedule:run
```

Run the scheduler every minute from cron. Scheduled tasks should be idempotent and protected against overlapping execution where necessary.

16. Storage: local and S3-compatible

VEDAIRO provides storage abstractions for local files and S3-compatible object storage. This allows application code to depend on a storage contract instead of a specific vendor.

```
S3_ENDPOINT=
S3_BUCKET=
S3_ACCESS_KEY=
S3_SECRET_KEY=
```

- Use object storage for user uploads at scale.
- Generate signed URLs for private files.
- Do not expose secret access keys to browser JavaScript.
- Configure lifecycle rules and backup/retention policies at the storage layer.

17. Multi-tenancy and audit

Tenant model

```
User
|
+--> Tenant
      |
      +--> Tenant-scoped data
```

Every tenant-scoped query must apply tenant isolation. Never trust a tenant ID sent by an untrusted client. Prefer deriving tenant context from the authenticated identity, verified host/subdomain mapping or another trusted resolver.

Audit

Audit logs should record security-sensitive and business-critical operations, including actor, action, resource, timestamp and relevant metadata. Avoid logging passwords, tokens or other secrets.

18. AI engine, providers, RAG and agents

Provider abstraction

The AI layer separates application code from provider-specific API formats. The release includes configuration/registry direction for OpenAI-compatible APIs, Gemini, Anthropic-compatible APIs and

Ollama-style local deployments.

```
AI_PROVIDER=openai
OPENAI_API_KEY=...
OPENAI_BASE_URL=https://api.openai.com/v1
OPENAI_MODEL=...
```

AI usage

```
$response = ai()->chat([
    ['role' => 'user',
     'content' => 'Write a product description']
]);
```

RAG

```
Documents
-> Chunking
-> Metadata
-> Retrieval
-> Context
-> LLM
-> Answer
```

The included RAG foundation supports stored documents/chunks and lexical retrieval without requiring an external vector service. For semantic search, implement the embedding contract with a configured embedding provider and compatible vector store.

AI agents

Expose explicit, permission-checked tools. Never give an LLM unrestricted PHP execution, shell access or unrestricted SQL. Tool arguments must be validated and authorization must be enforced independently of the model.

AI governance

- Track provider/model usage.
- Apply per-user and per-tenant budgets.
- Rate-limit expensive operations.
- Keep provider secrets server-side.
- Log enough metadata for operational auditing without storing sensitive prompt data unnecessarily.

19. Real-time SSE and metrics

Server-Sent Events

SSE provides one-way server-to-browser streaming suitable for progress updates, notifications and AI streaming-style experiences. For bidirectional applications, use a WebSocket-capable infrastructure extension.

Metrics

- Request IDs make distributed logs easier to correlate.
- Health endpoints support deployment checks.
- Track queue failures, latency, payment errors and AI usage.
- Use external monitoring/observability systems for production alerting.

20. Internationalization

The i18n foundation separates translated strings from application code. For a multilingual application, establish supported locales, timezone rules, currency formatting and translation fallback

behavior.

```
resources/lang/  
├─ en/  
├─ bn/  
└─ hi/
```

Store dates/times consistently, preferably in UTC at the persistence layer, and convert to the user's configured timezone for presentation.

21. Backup, maintenance and production deployment

Production checklist

1. Point DocumentRoot to public/
2. Configure .env / secret manager
3. Create least-privilege DB user
4. Run migrations
5. Configure HTTPS
6. Configure secure cookies
7. Configure Redis if selected
8. Configure SMTP
9. Configure S3/object storage if selected
10. Configure payment credentials and verified webhooks
11. Configure AI provider and budgets
12. Run tests
13. Start queue worker
14. Schedule scheduler every minute
15. Configure backups
16. Test restore
17. Disable APP_DEBUG
18. Remove/disable public/install.php
19. Enable monitoring
20. Run staging integration tests before live traffic

Backup

```
php vedairo backup storage/backups/backup.sql
```

A backup is only useful if it can be restored. Perform periodic restore drills and verify backup integrity.

22. Security checklist

Control	Production requirement
HTTPS	Required
APP_DEBUG	false
.env	Never commit or expose
Database	Least privilege
Passwords	Strong hashing
CSRF	Enabled for browser state changes
SQL	Parameterized queries
Uploads	Validated and safely stored
Authentication	Rate limited; 2FA for privileged users
Payments	Verified webhooks + idempotency

Control	Production requirement
AI tools	Allow-list + authorization
Cookies	Secure + HttpOnly + SameSite
Backups	Automated + restore tested

Security is a system property, not a single library. Review application code, server configuration, dependency updates, credentials, access policies and external-provider settings together.

23. Testing and troubleshooting

Framework test command

```
php vedairo test
```

Common problems

Problem	Check first
404 on every route	Web root, rewrite rules, public/index.php
Database connection error	.env DB_* values, server and credentials
Session not persisting	storage permissions, cookie settings, HTTPS/proxy
AJAX 419/403 style failure	CSRF token and session
Queue does nothing	queue configuration, worker process, failed jobs
Payment marked incorrectly	Webhook signature, idempotency and provider status
Mail not delivered	SMTP credentials, TLS, sender DNS and provider logs
Redis errors	Host/port/password/TLS/network ACL
AI request fails	Provider key, endpoint, model, quotas and network
Uploads fail	PHP limits, permissions, MIME/size validation, storage config

External integration testing

The framework cannot truthfully certify a live payment gateway, SMTP provider, Redis server, S3 account or AI service without access to that external system. Use provider sandbox/test credentials and run integration tests in staging.

24. Developer workflow and examples

Recommended module workflow

```
Database design
-> Migration
-> Model / Repository
-> Service / Business logic
-> Validation
-> Controller
-> Routes
-> View / API resource
-> AJAX
-> Pagination / Search
-> Authorization
-> Tests
-> Documentation
-> Deployment
```

Example migration concept

```
CREATE TABLE products (  
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(150) NOT NULL,  
  sku VARCHAR(100),  
  price DECIMAL(12,2) NOT NULL DEFAULT 0,  
  stock INT NOT NULL DEFAULT 0,  
  status TINYINT NOT NULL DEFAULT 1,  
  created_at TIMESTAMP NULL,  
  updated_at TIMESTAMP NULL  
);
```

Example paginated query

```
$products = Product::query()  
  ->whereEq('status', 1)  
  ->orderBy('id', 'DESC')  
  ->paginate(20, $page, '/products');
```

Principles

- Keep provider-specific code behind adapters.
- Keep business rules in services/domain operations.
- Validate every untrusted input.
- Authorize every protected operation.
- Write tests before calling a module production-ready.

25. Feature/status matrix and ecosystem boundaries

The v5.0.0 source tree is a consolidated enterprise foundation. The following matrix is intentionally honest: an “adapter/foundation” means framework-side support exists, while live external certification requires the operator's environment.

Area	v5.0.0 position
Core / MVC / Router / DI	Implemented foundation
Database / Query Builder / Pagination	Implemented foundation
Sessions / Auth / Remember-me	Implemented foundation
TOTP 2FA	Implemented
RBAC	Implemented foundation
REST / API tokens / OpenAPI	Implemented foundation
CRUD / AJAX / Cart / Checkout	Implemented foundation
Tax / Coupon / Invoice	Implemented services
Payments	Adapters + webhook helpers; staging verification required
SMTP / Mail	Integration abstraction; provider configuration required
Redis / Queue	Framework adapters/worker; service required
S3-compatible storage	Adapter/configuration; service required
Multi-tenancy	Foundation; application-level isolation must be enforced
AI providers	Provider abstraction/configuration; credentials/models required

Area	v5.0.0 position
RAG	Storage + lexical retrieval foundation
AI agents/tools	Allow-list architecture; production tool policies required
SSE	Foundation
i18n	Foundation
Backups / metrics / audit	Foundation/implemented services
WebAuthn/Passkeys	Safe extension point; use maintained audited implementation

A-Z operational truth

VEDAIRO is designed so an enterprise application can be built around a single framework architecture, but external infrastructure remains external. A complete production deployment therefore consists of VEDAIRO + its configured database + optional Redis + storage + mail + payment + AI providers + monitoring + tested operational procedures.

Final release principle

Do not treat a feature as production-ready merely because its class or configuration exists. Validate it with real integration tests, security review, failure handling, monitoring and deployment-specific configuration.

VEDAIRO™ • Enterprise 5.0.0 • Copyright (c) 2026 Cloud Soft Web LLP

Developed at Cloud Soft Web Lab (WB, India) • Author & Principal Architect: Susanta Ghosh (CEO, Cloud Soft Web LLP)